Informe: Diseño de la base de datos

Generación automática de itinerarios turísticos

Proyecto Astarship

Equipo: HAVING Fe

Bases de Datos II · Ingeniería de Software
Curso 2026–2027

Resumen

El presente informe tiene como objetivo diseñar una base de datos para la gestión de la generación automática de itinerarios turísticos de una agencia de viajes. Se realizó un análisis de los requerimientos funcionales, no funcionales y de entorno del sistema, lo que permitió identificar con claridad los procesos de gestión de guías, turistas, atracciones, actividades, paquetes, itinerarios y las seis consultas analíticas que constituyen el criterio de evaluación. A partir de ello se elaboró un Modelo Conceptual Relacional-Extendido (MERX, versión 1.8.1) que muestra las entidades, atributos, interrelaciones y agregaciones críticas, junto con la especificación de las restricciones de integridad y el esquema lógico relacional derivado. Finalmente, se valoró la corrección del diseño mediante la extracción del cubrimiento minimal de dependencias funcionales, la descomposición en Tercera Forma Normal (3FN) y la comprobación de las propiedades de preservación de dependencias (PPDF) y de join sin pérdida (PLJ). Se concluye que el diseño es coherente y se recogen, de forma explícita, los puntos abiertos que deben cerrarse antes de la implementación física.

Índice

1. Introducción **4**

2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema **4**

2.1. Requerimientos Funcionales 4
2.1.1. Actividades y atracciones 4
2.1.2. Paquetes turísticos, guías y turistas 5
2.1.3. Generación de itinerarios 5
2.1.4. Validación 5
2.1.5. Asignación y recorrido 5
2.1.6. Solicitud de modificación 6
2.1.7. Red de atracciones 6
2.1.8. Reportes y exportación 6
2.1.9. Consultas (criterio de evaluación) 6
2.2. Requerimientos No Funcionales 7
2.3. Requerimientos de Entorno 7

3. Modelo Conceptual Relacional-Extendido (MERX) **8**

3.1. Entidades 8
3.2. Interrelaciones 9
3.3. Agregaciones 10
3.4. Esquema lógico relacional 11
3.5. Red de atracciones y búsqueda de alternativas (RF-33) 12
3.5.1. Algoritmo base: Dijkstra 12
3.5.2. Intercambio del algoritmo con el patrón Strategy 13
3.5.3. Endpoint y flujo de la consulta 14

4. Especificación de las restricciones de integridad **15**

4.1. Restricciones de integridad establecidas y/o detectadas 15
4.2. Llaves, referencias, dominios y nulabilidad 15
4.3. Restricciones de Cardinalidad por Interrelación 18
4.3.1. Tourist – TravelGroup (Belongs) 18
4.3.2. Activity – Category (Classifies) 18
4.3.3. Activity – ActivityType (Is of Type) 18
4.3.4. Category – Package (Coverage) 18
4.3.5. Activity – Guide (Generates) 18
4.3.6. Activity – Attraction (At) 19
4.3.7. Guide – TravelGroup (Attends) 19
4.3.8. Itinerary – Package (Belongs) 19
4.3.9. Package – Admin (Generates) 19
4.3.10. Itinerary – Guide (Generates) 19
4.3.11. Admin – Assignment (Assigns) 19
4.3.12. Agregaciones 19

5. Valoración de la corrección del diseño de la base de datos **20**

5.1. Demostración formal de normalización (3FN), PPDF, PLJ y discusión del diseño . 20
5.1.1. Extracción de dependencias funcionales 20
5.1.2. Cubrimiento minimal 20
5.2. Descomposición formal en Tercera Forma Normal (3FN) 20
5.2.1. Aplicación del algoritmo 21
5.3. Garantía de cumplimiento de la PPDF y la PLJ 22

5.3.1. Preservación de dependencias funcionales (PPDF) 22
5.3.2. Join sin pérdida (PLJ) 22
5.4. Valoración final del diseño 22
5.4.1. Completitud respecto a los requerimientos 22
5.4.2. Observaciones y puntos abiertos detectados 22
5.5. Flexibilidad y extensibilidad 23

6. Conclusiones **23**

1. Introducción

Este informe tiene como propósito mostrar el proceso de análisis, diseño y valoración teórica de la base de datos correspondiente al proyecto «Astarship — generación automática de itinerarios turísticos», desarrollado por el equipo HAVING Fe en el marco de las asignaturas Bases de Datos II e Ingeniería de Software. En él se recoge el trabajo colaborativo orientado a establecer una especificación clara que garantice la correcta implementación de los procesos de almacenamiento, abarcando desde la gestión de los distintos actores (guías turísticos, jefe de operaciones, turistas y administrador) hasta la generación de itinerarios equilibrados bajo criterios de proporción por tipo, cobertura de categorías y cantidad total de actividades.

El documento inicia con un análisis y reformulación de los requerimientos funcionales, no funcionales (rendimiento, arquitectura desacoplada, patrones de diseño) y las restricciones técnicas y de entorno. Seguidamente se muestra el Modelo Conceptual Relacional-Extendido (MERX) para representar de forma estructurada las entidades, interrelaciones y agregaciones que conforman la lógica del proyecto, así como el esquema lógico relacional que se deriva de él. A continuación se detallan las restricciones de integridad detectadas, incluyendo la definición de llaves, referencias, dominios, nulabilidad y cardinalidades por interrelación. Finalmente se realiza una valoración del diseño mediante la extracción del cubrimiento minimal y su descomposición en 3FN, evaluando su completitud y pertinencia teórica, y se señalan las observaciones y puntos abiertos detectados durante el análisis.

2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema

Astarship es una aplicación web para una agencia de viajes que genera itinerarios turísticos de forma automática. El sistema administra el banco de actividades y atracciones, los paquetes turísticos en que se usan, los guías que las describen y validan, y los turistas que las recorren. La generación no es aleatoria: cada itinerario debe respetar criterios de equilibrio y conservar el orden de recorrido para poder analizar cómo se acumula el esfuerzo físico a lo largo del viaje. Quedan fuera del alcance pagos, facturación, reservas de transporte o alojamiento, mensajería entre usuarios e integraciones con servicios externos.

Actores del sistema

Actor	Qué hace en el sistema
Guía turístico	Registra atracciones y actividades; clasifica por categoría y tipo y asigna nivel de esfuerzo a las actividades que él describió; genera itinerarios; atiende solicitudes de modificación.
Jefe de operaciones turísticas	Revisa y aprueba los itinerarios generados; puede ordenar la confección de otro bajo sus criterios.
Turista	Consulta los itinerarios que le asignan, registra sus visitas, valora la experiencia y solicita modificaciones.
Administrador	Evita actividades duplicadas e impide la asignación doble de itinerarios (ver PA-1 en la sección 5.4).

Un mismo guía puede estar autorizado en unos paquetes y no en otros, y la autorización distingue entre **generar** y **validar** itinerarios.

2.1. Requerimientos Funcionales

2.1.1. Actividades y atracciones

ID	Requisito	Actor
RF-01	Registrar una atracción.	Guía
RF-02	Registrar una actividad e indicar en qué atracción se realiza.	Guía
RF-03	Clasificar una actividad por **categoría**, solo sobre las actividades que el propio guía describió.	Guía
RF-04	Clasificar una actividad por **tipo**: visita, excursión o taller.	Guía
RF-05	Asignar el **nivel de esfuerzo físico**: bajo, medio o alto.	Guía
RF-06	Impedir que se registren actividades duplicadas.	Administrador
RF-07	Consultar y listar el banco de actividades y atracciones.	Todos

2.1.2. Paquetes turísticos, guías y turistas

ID	Requisito	Actor
RF-08	Registrar un paquete con identificador único, nombre, destino y la lista de categorías a cubrir.	—
RF-09	Registrar en qué paquetes está autorizado cada guía, distinguiendo si puede generar, validar o ambas.	—
RF-10	Registrar un turista con identificador único, nombre, edad, grupo de viaje y los paquetes en que está inscrito.	—
RF-11	Registrar un guía con identificador único, nombre y especialidad.	—
RF-12	Gestionar grupos de viaje y la pertenencia de turistas a ellos.	—

2.1.3. Generación de itinerarios

ID	Requisito	Actor
RF-13	Generar un itinerario automáticamente a partir de un conjunto de actividades seleccionadas.	Guía autorizado
RF-14	Parametrizar la generación con la proporción de actividades por tipo, la cobertura de categorías y la cantidad total de actividades.	Guía autorizado
RF-15	Almacenar la parametrización junto al itinerario generado.	Sistema
RF-16	Conservar el orden en que las actividades serán recorridas.	Sistema
RF-17	Asociar cada itinerario a un paquete turístico.	Sistema
RF-18	Consultar los itinerarios de otros guías que atienden el mismo paquete.	Guía

El enunciado separa quién *añade* actividades de quién *crea* el itinerario: son roles distintos sobre la misma entidad Guía, no personas de distinto tipo.

2.1.4. Validación

ID	Requisito	Actor
RF-19	Revisar y aprobar o rechazar un itinerario generado, registrando fecha de validación, resultado y observaciones.	Jefe de operaciones
RF-20	Indicar la confección de otro itinerario bajo sus propios criterios.	Jefe de operaciones
RF-21	Distinguir el estado de un itinerario entre «generado pendiente de validación» y «validado».	Sistema

2.1.5. Asignación y recorrido

ID	Requisito	Actor
RF-22	Asignar un itinerario a un turista (actor por definir, ver PA-1).	—
RF-23	Impedir la asignación doble de itinerarios.	Administrador
RF-24	Consultar los itinerarios asignados.	Turista
RF-25	Registrar cada visita de un turista a una actividad de un itinerario recorrido.	Turista / Sistema
RF-26	Valorar la experiencia.	Turista
RF-27	Consultar las valoraciones.	Guía y Turista
RF-28	Consultar todas las valoraciones propias agrupadas por paquete turístico.	Turista

2.1.6. Solicitud de modificación

ID	Requisito	Actor
RF-29	Solicitar de forma virtual una modificación del itinerario.	Turista
RF-30	Pedir el apoyo de un guía adicional al asignado para que atienda la solicitud.	Turista
RF-31	Registrar todo el proceso de solicitud y el resultado de la modificación manual.	Sistema

2.1.7. Red de atracciones

ID	Requisito	Actor
RF-32	Mantener las conexiones entre atracciones según cercanía geográfica y transporte disponible, con tiempo y costo asociados.	Sistema
RF-33	Recorrer la red para proponer alternativas cuando una actividad no pueda realizarse en el horario previsto (Dijkstra con Strategy; ver sección 3.5).	Sistema

2.1.8. Reportes y exportación

ID	Requisito	Actor
RF-34	Generar reportes sobre la creación de itinerarios, las actividades más utilizadas, la distribución de niveles de esfuerzo y el desempeño de la experiencia según el esfuerzo.	Todos
RF-35	Presentar los resultados en tablas y gráficos.	Sistema
RF-36	Exportar a PDF la información mostrada, para todo tipo de usuario.	Todos
RF-37	Ordenar cada columna de los resultados.	Todos

2.1.9. Consultas (criterio de evaluación)

Técnicamente son requisitos funcionales; se separan porque constituyen el criterio de evaluación de Bases de Datos. Cada una debe poder verse como tabla y como gráfico, exportarse a PDF y ordenarse por columna (RF-35 a RF-37).

ID	Consulta
RC-1	Itinerarios generados automáticamente para un paquete, indicando creador, fecha de creación y parámetros utilizados.
RC-2	Actividades más utilizadas en los itinerarios finales de un paquete, clasificadas por nivel de esfuerzo y categoría.
RC-3	Itinerarios validados por un revisor determinado, con fecha de validación y observaciones.
RC-4	Experiencia de los turistas en un itinerario, clasificando las actividades por esfuerzo y comparando tasas de satisfacción.

ID	Consulta
RC-5	Comparación entre paquetes: distribución de actividades por categoría y esfuerzo, y cumplimiento de los criterios de equilibrio.
RC-6	Correlación entre nivel de esfuerzo y valoración promedio por paquete; las 10 actividades con mayor tasa de abandono (con su guía creador y su paquete); y la comparación del desempeño de los turistas que solicitaron modificación contra el promedio de sus grupos de viaje en esos mismos paquetes.

RC-6 es la consulta más exigente y condiciona el modelo: exige calcular una *tasa de abandono* por actividad (distinguir una visita realizada de una no realizada, RF-25), trazar cada actividad hasta el guía que la creó (RF-02) y relacionar solicitudes de modificación con valoraciones y grupos de viaje.

2.2. Requerimientos No Funcionales

ID	Requisito
RNF-01	**Rendimiento.** El itinerario del día debe entregarse sin demoras perceptibles, incluso con varios grupos consultando a la vez; justifica una capa de caché.
RNF-02	**La red de atracciones no se modela como tabla del esquema relacional.** Se gestiona de forma independiente para poder navegarla con agilidad.
RNF-03	**Multiplataforma.** Accesible desde cualquier sistema; una aplicación web es suficiente.
RNF-04	**Arquitectura desacoplada, extensible y mantenible.**
RNF-05	**Patrones de diseño.** Al menos dos implementados y justificados (Strategy para el algoritmo de rutas, sección 3.5).
RNF-06	**Documentación.** Docstrings en todo el código, verificados automáticamente.
RNF-07	**Pruebas unitarias** de back-end y front-end.
RNF-08	**Control de versiones** (Git + GitHub).
RNF-09	**Planificación** con herramienta CASE.
RNF-10	**Autenticación y autorización por rol.** Cada actor ve y hace solo lo que le corresponde.

2.3. Requerimientos de Entorno

Decisiones técnicas ya tomadas por el equipo y aprobadas por el profesor:

- **RE-01 Backend:** Python + FastAPI, gestionado con `uv`.
- **RE-02 Base de datos:** PostgreSQL 18, con SQLAlchemy y migraciones con Alembic.
- **RE-03 Frontend:** TypeScript + React (Vite).
- **RE-04 Entorno:** Docker y Docker Compose.
- **RE-05 Despliegue:** Vercel (frontend) + Render (API);
- **RE-06 Caché:** Redis,

3. Modelo Conceptual Relacional-Extendido (MERX)

El MERX (versión 1.8.1) se organiza en tres bloques: entidades con sus atributos, interrelaciones binarias y agregaciones que modelan las interrelaciones con atributos propios o que participan en otras interrelaciones.

3.1. Entidades

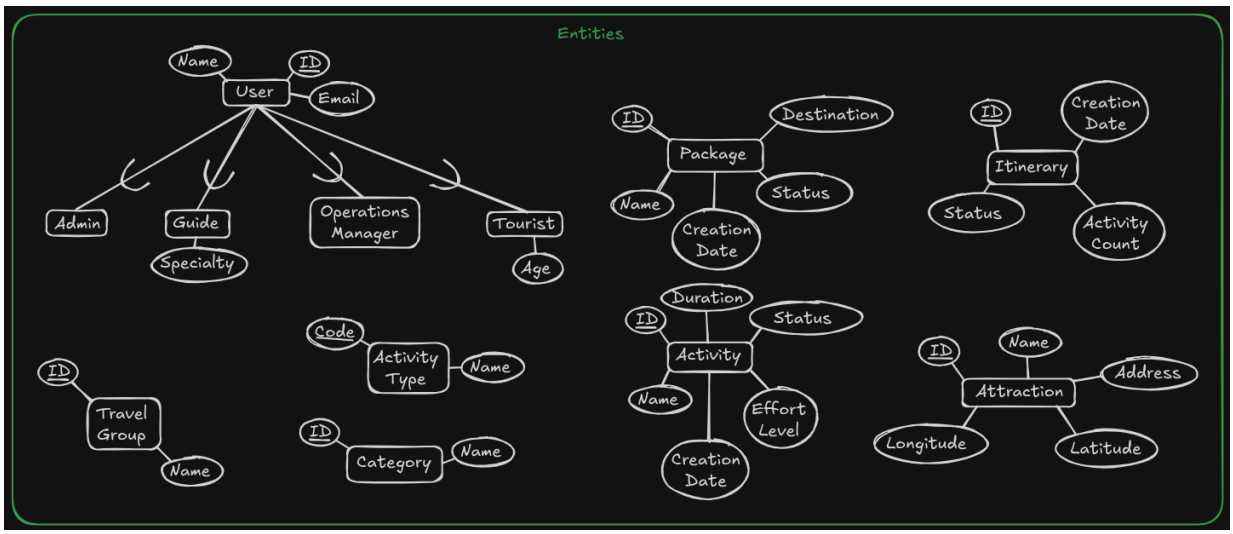


Figura 1: MERX v1.8.1 — Entidades.

Entidad	Atributos
User	ID, Name, Email
TravelGroup	ID, Name
Package	ID, Name, Destination, Status, CreationDate
Itinerary	ID, CreationDate, Status, ActivityCount
ActivityType	Code, Name
Category	ID, Name
Activity	ID, Name, EffortLevel, Duration, Status, CreationDate
Attraction	ID, Name, Address, Latitude, Longitude

Especialización de User (total, disjunta — partición):

Supertipo	Subtipo	Atributos propios
User	Admin	—
User	Guide	Specialty
User	OperationsManager	—
User	Tourist	Age

3.2. Interrelaciones

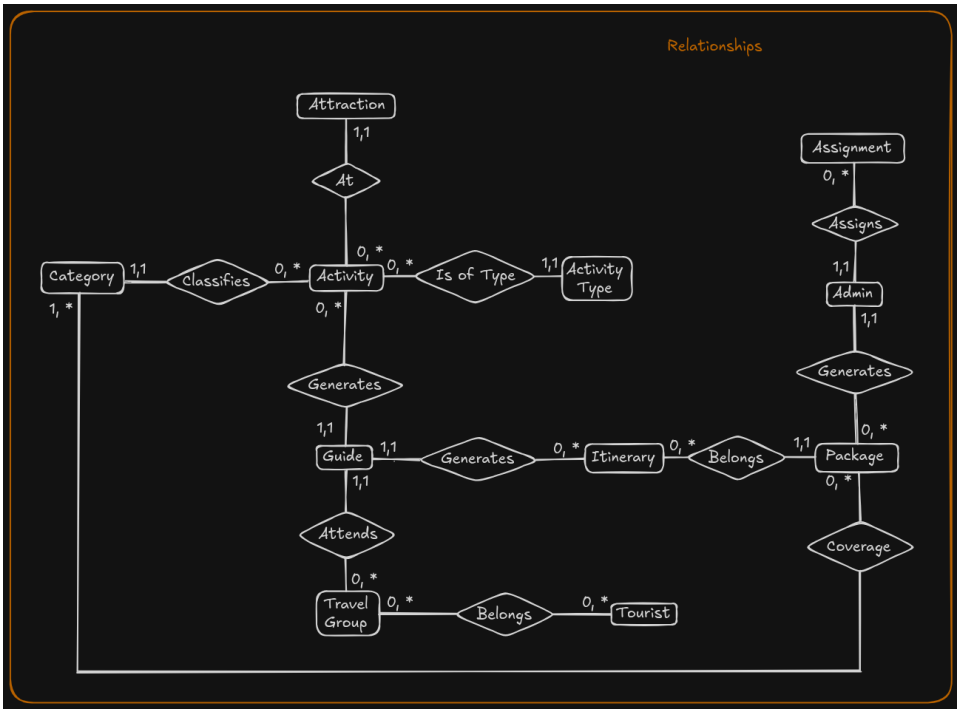


Figura 2: MERX v1.8.1 — Interrelaciones.

Interrelación	**Participantes**	**Cardinalidad**	**(min,max)**
Belongs	Tourist — TravelGroup	N:1	Tourist (1,1) — TravelGroup (1,*)
Classifies	Activity — Category	N:1	Activity (1,1) — Category (0,*)
Is of Type	Activity — ActivityType	N:1	Activity (1,1) — ActivityType (0,*)
Coverage	Category — Package	N:M	Category (0,*) — Package (1,*)
Generates	Activity — Guide	N:1	Activity (1,1) — Guide (0,*)
At	Activity — Attraction	N:1	Activity (1,1) — Attraction (0,*)
Attends	Guide — TravelGroup	1:N	Guide (0,*) — TravelGroup (1,1)
Belongs	Itinerary — Package	N:1	Itinerary (1,1) — Package (0,*)
Generates	Package — Admin	N:1	Package (1,1) — Admin (0,*)
Generates	Itinerary — Guide	N:1	Itinerary (1,1) — Guide (0,*)
Assigns	Admin — Assignment	1:N	Admin (0,*) — Assignment (1,1)

3.3. Agregaciones

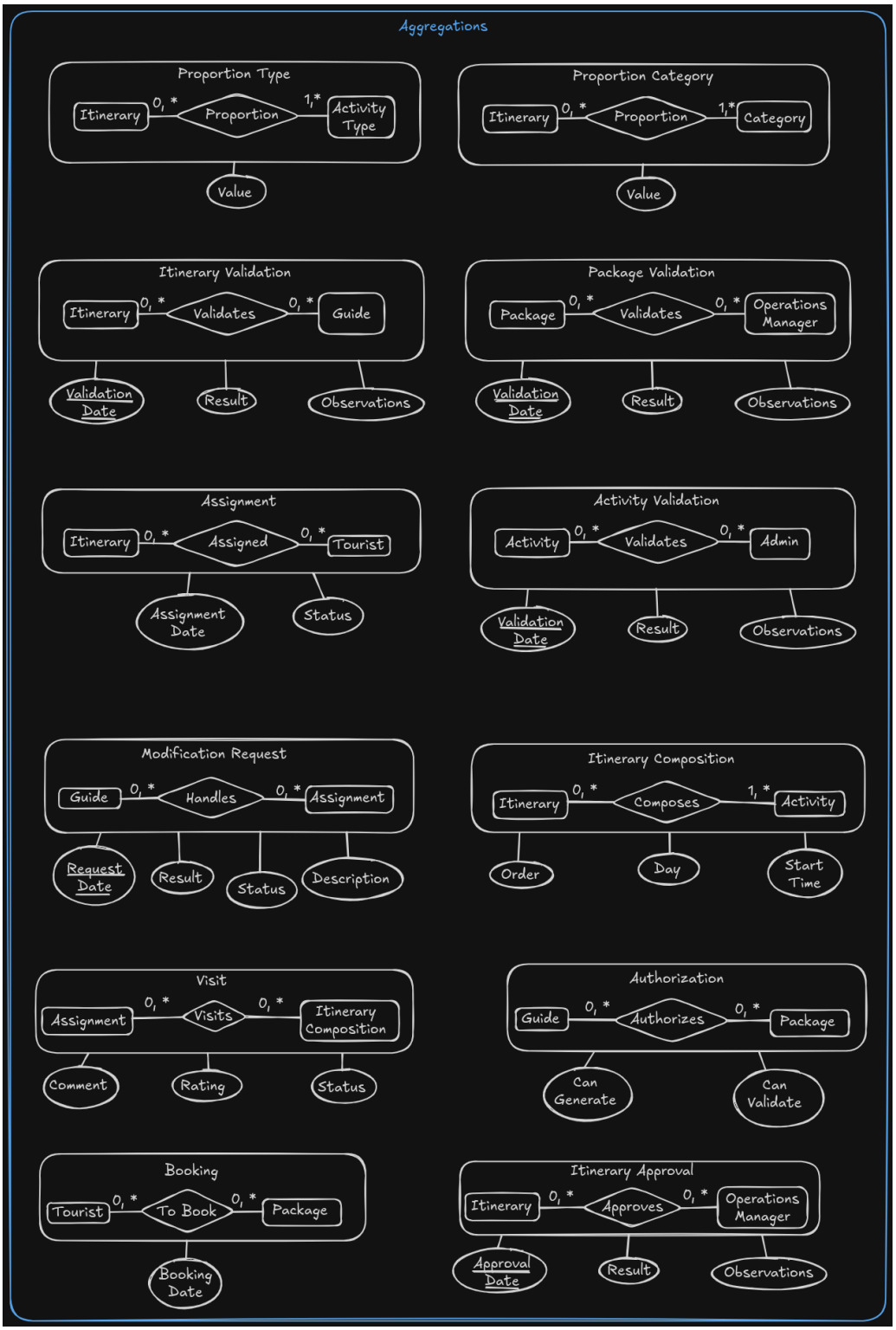


Figura 3: MERX v1.8.1 — Agregaciones.

Agregación	Participantes	Card.	(min,max)	Atributos
Proportion Type	Itinerary — ActivityType	N:M	Itinerary (1,*) — ActivityType (0,*)	Value

Agregación	Participantes	Card.	(min,max)	Atributos
Proportion Category	Itinerary — Category	N:M	Itinerary (1,*) — Category (0,*)	Value
Itinerary Validation	Itinerary — Guide	N:M	Itinerary (0,*) — Guide (0,*)	ValidationDate, Result, Observations
Itinerary Approval	Itinerary — OperationsManager	N:M	Itinerary (0,*) — OperationsManager (0,*)	ApprovalDate, Result, Observations
Package Validation	Package — OperationsManager	N:M	Package (0,*) — OperationsManager (0,*)	ValidationDate, Result, Observations
Activity Validation	Activity — Admin	N:M	Activity (0,*) — Admin (0,*)	ValidationDate, Result, Observations
Assignment	Itinerary — Tourist	N:M	Itinerary (0,*) — Tourist (0,*)	AssignmentDate, Status
Modification Request	Guide — Assignment	N:M	Guide (0,*) — Assignment (0,*)	RequestDate, Description, Status, Result
Itinerary Composition	Itinerary — Activity	N:M	Itinerary (1,*) — Activity (0,*)	Order, Day, StartTime
Visit	Assignment — Itinerary Composition	N:M	Assignment (0,*) — Itinerary Composition (0,*)	Comment, Rating, Status
Authorization	Guide — Package	N:M	Guide (0,*) — Package (0,*)	CanGenerate, CanValidate
Booking	Tourist — Package	N:M	Tourist (0,*) — Package (0,*)	BookingDate

Las agregaciones *Assignment* e *Itinerary Composition* participan a su vez como extremos de otras interrelaciones (*Modification Request* y *Visit*), por lo que se tratan como entidades agregadas con identidad propia dada por la concatenación de las llaves de sus participantes.

3.4. Esquema lógico relacional

Aplicando las reglas de transformación del MERX al modelo relacional se obtienen 25 esquemas: las entidades incorporan como llave foránea las interrelaciones de cardinalidad N:1 / 1:N; cada subtipo de la especialización comparte la llave de User (partición total y disjunta); la interrelación N:M Coverage y las agregaciones generan esquemas propios cuya llave es la concatenación de las llaves de sus participantes.

Relación	Tipo	Atributos (llave subrayada)	Llaves foráneas
User	Entidad	ID, Name, Email	—
TravelGroup	Entidad	ID, Name, ID_Guide	(ID_Guide) → Guide(ID)
Package	Entidad	ID, Name, Destination, Status, CreationDate, ID_Admin	(ID_Admin) → Admin(ID)
Itinerary	Entidad	ID, CreationDate, Status, ActivityCount, ID_Package, ID_Guide	(ID_Package) → Package(ID) (ID_Guide) → Guide(ID)
ActivityType	Entidad	Code, Name	—
Category	Entidad	ID, Name	—
Attraction	Entidad	ID, Name, Address, Latitude, Longitude	—
Activity	Entidad	ID, Name, EffortLevel, Duration, Status, CreationDate, Code_Type, ID_Category, ID_Guide, ID_Attraction	(Code_Type) → ActivityType(Code) (ID_Category) → Category(ID) (ID_Guide) → Guide(ID) (ID_Attraction) → Attraction(ID)
Admin	Especialización	ID	(ID) → User(ID)
Guide	Especialización	ID, Specialty	(ID) → User(ID)
OperationsManager	Especialización	ID	(ID) → User(ID)

Relación	Tipo	Atributos (llave subrayada)	Llaves foráneas
Tourist	Especialización	ID, Age, ID_TravelGroup	(ID) → User(ID) (ID_TravelGroup) → TravelGroup(ID)
Coverage	Interrelación	ID_Package, ID_Category	(ID_Package) → Package(ID) (ID_Category) → Category(ID)
ProportionType	Agregación	ID_Itinerary, Code_Type, Value	(ID_Itinerary) → Itinerary(ID) (Code_Type) → ActivityType(Code)
ProportionCategory	Agregación	ID_Itinerary, ID_Category, Value	(ID_Itinerary) → Itinerary(ID) (ID_Category) → Category(ID)
ItineraryValidation	Agregación	ID_Itinerary, ID_Guide, ValidationDate, Result, Observations	(ID_Itinerary) → Itinerary(ID) (ID_Guide) → Guide(ID)
ItineraryApproval	Agregación	ID_Itinerary, ID_Manager, ApprovalDate, Result, Observations	(ID_Itinerary) → Itinerary(ID) (ID_Manager) → OperationsManager(ID)
PackageValidation	Agregación	ID_Package, ID_Manager, ValidationDate, Result, Observations	(ID_Package) → Package(ID) (ID_Manager) → OperationsManager(ID)
ActivityValidation	Agregación	ID_Activity, ID_Admin, ValidationDate, Result, Observations	(ID_Activity) → Activity(ID) (ID_Admin) → Admin(ID)
Assignment	Agregación	ID_Itinerary, ID_Tourist, AssignmentDate, Status, ID_Admin	(ID_Itinerary) → Itinerary(ID) (ID_Tourist) → Tourist(ID) (ID_Admin) → Admin(ID)
ModificationRequest	Agregación	ID_Itinerary, ID_Tourist, ID_Guide, RequestDate, Description, Status, Result	(ID_Itinerary, ID_Tourist) → Assignment(ID_Itinerary, ID_Tourist) (ID_Guide) → Guide(ID)
ItineraryComposition	Agregación	ID_Itinerary, ID_Activity, Order, Day, StartTime	(ID_Itinerary) → Itinerary(ID) (ID_Activity) → Activity(ID)
Visit	Agregación	ID_Itinerary, ID_Tourist, ID_Activity, Comment, Rating, Status	(ID_Itinerary, ID_Tourist) → Assignment(ID_Itinerary, ID_Tourist) (ID_Itinerary, ID_Activity) → ItineraryComposition(ID_Itinerary, ID_Activity)
Authorization	Agregación	ID_Guide, ID_Package, CanGenerate, CanValidate	(ID_Guide) → Guide(ID) (ID_Package) → Package(ID)
Booking	Agregación	ID_Tourist, ID_Package, BookingDate	(ID_Tourist) → Tourist(ID) (ID_Package) → Package(ID)

Conforme a RNF-02, la red de atracciones (tiempos y costos entre atracciones) **no** forma parte de este esquema relacional: se gestiona de forma independiente y se enlaza con Attraction mediante su identificador.

3.5. Red de atracciones y búsqueda de alternativas (RF-33)

RF-33 exige recorrer la red de atracciones para proponer alternativas cuando una actividad no puede realizarse en el horario previsto. Por RNF-02 la red no vive en el esquema relacional: cada *nodo* es una atracción (identificada por `Attraction.ID`, con `Latitude` y `Longitude`) y cada *arista* es una conexión dirigida con dos pesos, **tiempo** y **costo** (RF-32). Esta subsección describe el algoritmo elegido y su integración en el sistema.

3.5.1. Algoritmo base: Dijkstra

Dijkstra encuentra el camino de menor peso acumulado entre un origen y un destino en un grafo dirigido con pesos no negativos. Mantiene una cola de prioridad ordenada por $g(n)$, el peso acumulado desde el origen, y en cada paso extrae el nodo de menor $g(n)$ y relaja sus aristas

salientes. Como los pesos (tiempo y costo, RF-32) son no negativos, la primera vez que un nodo sale de la cola su peso es el óptimo, por lo que la búsqueda puede detenerse al extraer el destino.

- **Peso por tiempo y peso por costo.** El mismo algoritmo sirve para ambos criterios: solo cambia la función *peso(arista)* que se le entrega (el tiempo o el costo de la conexión). No requiere coordenadas ni heurística, por lo que no depende de `Latitude` y `Longitude` de Attraction.
- **Alternativas múltiples.** Una única ejecución desde la atracción de origen calcula el peso mínimo hacia *todas* las atracciones alcanzables, de modo que el servicio puede evaluar todas las atracciones candidatas con una sola pasada en lugar de una búsqueda por candidata.
- **Complejidad.** O((V + E) log V) con cola de prioridad binaria, siendo V el número de atracciones y E el de conexiones.

```
DIJKSTRA(grafo, origen, destino, peso):
    abiertos <- cola_prioridad();  g[origen] <- 0
    abiertos.insertar(origen, prioridad = 0)
    mientras abiertos no vacia:
        n <- abiertos.extraer_minimo()
        si n == destino: devolver reconstruir_camino(padre, n)
        para cada arista (n, m, w) en grafo.vecinos(n):
            nuevo <- g[n] + peso(w)             # tiempo o costo
            si nuevo < g.get(m, infinito):
                g[m] <- nuevo;  padre[m] <- n
                abiertos.insertar_o_actualizar(m, nuevo)
    devolver SIN_CAMINO
```

3.5.2. Intercambio del algoritmo con el patrón Strategy

El algoritmo de búsqueda y su criterio de optimización (tiempo o costo) se encapsulan tras una interfaz común, de modo que el servicio que propone alternativas no conoce el algoritmo concreto y puede cambiarse sin modificarlo. Dijkstra es la estrategia por defecto, pero el patrón se mantiene para poder incorporar otras búsquedas (por ejemplo A*, búsqueda bidireccional o BFS por número de transbordos) sin alterar el resto del sistema. Este es uno de los dos patrones de diseño exigidos por RNF-05, junto con el que se elija y justifique en el resto de la aplicación.

Rol (Strategy)	**Elemento**	**Responsabilidad**
Strategy	`RouteStrategy` (interfaz)	Declara `find_path(graph, origin, target)` → `Route | None`.
ConcreteStrategy	`DijkstraTimeStrategy`	Dijkstra con peso = tiempo (estrategia por defecto).
ConcreteStrategy	`DijkstraCostStrategy`	Dijkstra con peso = costo.
ConcreteStrategy (futura)	`OtraBusquedaStrategy`	Cualquier otro algoritmo que implemente `find_path` (A*, bidireccional, BFS, etc.).
Context	`AlternativesService`	Recibe la estrategia por inyección, filtra candidatos y ordena resultados.
Cliente	Endpoint FastAPI	Elige la estrategia según el parámetro de criterio y se la entrega al servicio.

```
class RouteStrategy(Protocol):
    def find_path(self, graph: AttractionGraph, origin: int, target: int) -> Route | None: ...

class DijkstraTimeStrategy:   # Dijkstra, peso = tiempo (por defecto)
class DijkstraCostStrategy:   # Dijkstra, peso = costo
# class AStarStrategy: ...    # ejemplo de extensión futura: otra búsqueda, misma interfaz

class AlternativesService:
    def __init__(self, strategy: RouteStrategy): self._strategy = strategy
```

Añadir un nuevo criterio o un algoritmo distinto de Dijkstra (por ejemplo, A*, o minimizar transbordos) solo requiere una nueva estrategia concreta, sin tocar el servicio ni el endpoint, lo que refuerza RNF-04 (arquitectura extensible y mantenible).

3.5.3. Endpoint y flujo de la consulta

```
GET /attractions/{id}/alternatives?activity=<id>&time=<hora>
```

Parámetro	Significado
`id` (ruta)	Atracción donde la actividad no puede realizarse; es el *origen* de la búsqueda.
`activity`	Actividad afectada; se usa para buscar actividades sustitutas comparables (misma categoría y tipo).
`time`	Horario previsto en que la actividad no puede realizarse; define la ventana en que la alternativa debe estar disponible.

- Se obtienen las actividades candidatas en el esquema relacional (Activity con la misma Category y ActivityType, y su Attraction asociada).
- Para cada atracción candidata se ejecuta la estrategia desde la atracción de origen; si el criterio es tiempo, se descartan las que no llegan antes de que termine la ventana del horario indicado.
- Se ordenan las alternativas por peso del camino (tiempo o costo) y se devuelven con la ruta calculada y su peso total.
- Si no hay camino o ninguna candidata es alcanzable, se responde con una lista vacía, no con un error.

Supuestos que debe confirmar el equipo: los nombres y la semántica exacta de `activity` y `time`, la selección del criterio (tiempo o costo) mediante un parámetro adicional, y la forma de la respuesta no están fijados en el enunciado ni en `requisitos.md`; aquí se proponen como diseño inicial. Dado que RNF-01 exige respuestas sin demora perceptible, el grafo se mantiene en memoria (con caché si hace falta) y no se consulta la base de datos en cada expansión de nodo.

4. Especificación de las restricciones de integridad

4.1. Restricciones de integridad establecidas y/o detectadas

Se detallan las reglas y restricciones que aseguran la fiabilidad de los datos dentro del sistema, garantizando que el modelo refleje correctamente las reglas de la agencia.

- **RN-01.** Un guía solo clasifica (categoría, tipo, esfuerzo) las actividades que él describió, es decir, aquellas donde `Activity.ID_Guide` coincide con su identificador.
- **RN-02.** Un guía solo genera o valida itinerarios de paquetes en los que está autorizado y según el tipo de autorización (`CanGenerate`, `CanValidate` en Authorization).
- **RN-03.** No pueden existir actividades duplicadas (unicidad sobre el nombre de la actividad dentro de una misma atracción).
- **RN-04.** Un turista no puede recibir una asignación doble; la llave de Assignment (Itinerary, Tourist) impide asignar dos veces el mismo itinerario (ver PA-3).
- **RN-05.** Un itinerario pertenece a un único paquete; un paquete puede tener varios itinerarios.
- **RN-06.** El orden de las actividades dentro de un itinerario es significativo y debe preservarse (atributos Order y Day de Itinerary Composition).
- **RN-07.** Un itinerario generado no es válido hasta que el jefe de operaciones lo aprueba.
- **RN-08.** El guía que atiende una solicitud de modificación puede ser distinto del guía asignado al grupo del turista.
- Si el resultado de una validación o aprobación es RECHAZADO, el atributo Observations es obligatorio; si es APROBADO puede estar vacío.
- La suma de las proporciones por tipo (ProportionType) y por categoría (ProportionCategory) de un itinerario debe ser coherente con su cantidad total de actividades.
- El atributo ActivityCount de un itinerario debe coincidir con el número de filas de Itinerary Composition de ese itinerario.
- Los valores de Rating en Visit deben estar restringidos a un rango numérico predefinido (p. ej., 1 a 5).
- Solo se puede registrar una visita (Visit) a una actividad que forme parte del itinerario asignado al turista; la clave foránea compuesta sobre Itinerary Composition lo garantiza.
- Un itinerario solo puede asignarse a un turista si su estado es «validado».

4.2. Llaves, referencias, dominios y nulabilidad

Llaves primarias

- `PK_User`: (ID).
- `PK_TravelGroup`: (ID).
- `PK_Package`: (ID).
- `PK_Itinerary`: (ID).
- `PK_ActivityType`: (Code).
- `PK_Category`: (ID).
- `PK_Attraction`: (ID).
- `PK_Activity`: (ID).

- `PK_Admin`: (ID).
- `PK_Guide`: (ID).
- `PK_OperationsManager`: (ID).
- `PK_Tourist`: (ID).
- `PK_Coverage`: (ID_Package, ID_Category).
- `PK_ProportionType`: (ID_Itinerary, Code_Type).
- `PK_ProportionCategory`: (ID_Itinerary, ID_Category).
- `PK_ItineraryValidation`: (ID_Itinerary, ID_Guide, ValidationDate).
- `PK_ItineraryApproval`: (ID_Itinerary, ID_Manager, ApprovalDate).
- `PK_PackageValidation`: (ID_Package, ID_Manager, ValidationDate).
- `PK_ActivityValidation`: (ID_Activity, ID_Admin, ValidationDate).
- `PK_Assignment`: (ID_Itinerary, ID_Tourist).
- `PK_ModificationRequest`: (ID_Itinerary, ID_Tourist, ID_Guide, RequestDate).
- `PK_ItineraryComposition`: (ID_Itinerary, ID_Activity).
- `PK_Visit`: (ID_Itinerary, ID_Tourist, ID_Activity).
- `PK_Authorization`: (ID_Guide, ID_Package).
- `PK_Booking`: (ID_Tourist, ID_Package).

Llaves foráneas

- `FK_TravelGroup_ID_Guide`: (ID_Guide) referencia a Guide(ID).
- `FK_Package_ID_Admin`: (ID_Admin) referencia a Admin(ID).
- `FK_Itinerary_ID_Package`: (ID_Package) referencia a Package(ID).
- `FK_Itinerary_ID_Guide`: (ID_Guide) referencia a Guide(ID).
- `FK_Activity_Code_Type`: (Code_Type) referencia a ActivityType(Code).
- `FK_Activity_ID_Category`: (ID_Category) referencia a Category(ID).
- `FK_Activity_ID_Guide`: (ID_Guide) referencia a Guide(ID).
- `FK_Activity_ID_Attraction`: (ID_Attraction) referencia a Attraction(ID).
- `FK_Admin_ID`: (ID) referencia a User(ID).
- `FK_Guide_ID`: (ID) referencia a User(ID).
- `FK_OperationsManager_ID`: (ID) referencia a User(ID).
- `FK_Tourist_ID`: (ID) referencia a User(ID).
- `FK_Tourist_ID_TravelGroup`: (ID_TravelGroup) referencia a TravelGroup(ID).
- `FK_Coverage_ID_Package`: (ID_Package) referencia a Package(ID).
- `FK_Coverage_ID_Category`: (ID_Category) referencia a Category(ID).
- `FK_ProportionType_ID_Itinerary`: (ID_Itinerary) referencia a Itinerary(ID).
- `FK_ProportionType_Code_Type`: (Code_Type) referencia a ActivityType(Code).

- FK_ProportionCategory_ID_Itinerary: (ID_Itinerary) referencia a Itinerary(ID).
- FK_ProportionCategory_ID_Category: (ID_Category) referencia a Category(ID).
- FK_ItineraryValidation_ID_Itinerary: (ID_Itinerary) referencia a Itinerary(ID).
- FK_ItineraryValidation_ID_Guide: (ID_Guide) referencia a Guide(ID).
- FK_ItineraryApproval_ID_Itinerary: (ID_Itinerary) referencia a Itinerary(ID).
- FK_ItineraryApproval_ID_Manager: (ID_Manager) referencia a OperationsManager(ID).
- FK_PackageValidation_ID_Package: (ID_Package) referencia a Package(ID).
- FK_PackageValidation_ID_Manager: (ID_Manager) referencia a OperationsManager(ID).
- FK_ActivityValidation_ID_Activity: (ID_Activity) referencia a Activity(ID).
- FK_ActivityValidation_ID_Admin: (ID_Admin) referencia a Admin(ID).
- FK_Assignment_ID_Itinerary: (ID_Itinerary) referencia a Itinerary(ID).
- FK_Assignment_ID_Tourist: (ID_Tourist) referencia a Tourist(ID).
- FK_Assignment_ID_Admin: (ID_Admin) referencia a Admin(ID).
- FK_ModificationRequest_ID_Itinerary_ID_Tourist: (ID_Itinerary, ID_Tourist) referencia a Assignment(ID_Itinerary, ID_Tourist).
- FK_ModificationRequest_ID_Guide: (ID_Guide) referencia a Guide(ID).
- FK_ItineraryComposition_ID_Itinerary: (ID_Itinerary) referencia a Itinerary(ID).
- FK_ItineraryComposition_ID_Activity: (ID_Activity) referencia a Activity(ID).
- FK_Visit_ID_Itinerary_ID_Tourist: (ID_Itinerary, ID_Tourist) referencia a Assignment(ID_Itinerary, ID_Tourist).
- FK_Visit_ID_Itinerary_ID_Activity: (ID_Itinerary, ID_Activity) referencia a ItineraryComposition(ID_Itinerary, ID_Activity).
- FK_Authorization_ID_Guide: (ID_Guide) referencia a Guide(ID).
- FK_Authorization_ID_Package: (ID_Package) referencia a Package(ID).
- FK_Booking_ID_Tourist: (ID_Tourist) referencia a Tourist(ID).
- FK_Booking_ID_Package: (ID_Package) referencia a Package(ID).

Dominios

- Dom_EffortLevel_Activity: el nivel de esfuerzo solo puede ser {bajo, medio, alto}.
- Dom_Name_ActivityType: el tipo de actividad solo puede ser {visita, excursión, taller}.
- Dom_Status_Itinerary: el estado de un itinerario solo puede ser {generado pendiente de validación, validado}.
- Dom_Result_Validation: el resultado de una validación o aprobación solo puede ser {aprobado, rechazado}.
- Dom_Rating_Visit: la valoración es un entero en el rango definido por el cliente (p. ej., 1 a 5).
- Dom_Status_Visit: el estado de una visita distingue {realizada, no realizada/abandonada}; es la base de la tasa de abandono de RC-6.
- Dom_Status_Assignment, Dom_Status_ModificationRequest, Dom_Status_Package, Dom_Status_Activity: conjuntos de estados {(definido por el equipo)}.

- `Dom_Specialty_Guide`: especialidades de un guía {(definido por el cliente)}.
- `Dom_Age_Tourist`: la edad es un entero positivo.
- `Dom_Duration_Activity`: la duración es un valor positivo (en minutos).
- `Dom_Latitude/Longitude_Attraction`: latitud en $[-90, 90]$ y longitud en $[-180, 180]$.
- `Dom_Value_Proportion`: la proporción es un valor en $[0, 1]$ (o porcentaje en $[0, 100]$).
- `Dom_Day_ItineraryComposition`: el día es un entero positivo; `Order` es un entero positivo, único por itinerario y día.

Nulabilidad

- `NULL_Observations`: el campo Observations de ItineraryValidation, ItineraryApproval, PackageValidation y ActivityValidation es opcional (anulable), salvo cuando el resultado es «rechazado».
- `NULL_Comment_Visit`: el comentario de una visita es opcional.
- `NULL_Rating_Visit`: la valoración es anulable mientras la visita no haya sido realizada.
- `NULL_Result_ModificationRequest`: el resultado de una solicitud de modificación es nulo mientras esté pendiente de atención.
- Los restantes atributos de todos los esquemas son obligatorios (NOT NULL), incluidas todas las llaves foráneas de las relaciones derivadas de cardinalidad mínima 1.

4.3. Restricciones de Cardinalidad por Interrelación

4.3.1. Tourist – TravelGroup (Belongs)

- **Tourist** (1,1): cada Tourist se asocia con exactamente un(a) TravelGroup.
- **TravelGroup** (1,*): cada TravelGroup se asocia con de uno a muchos(as) Tourist.

4.3.2. Activity – Category (Classifies)

- **Activity** (1,1): cada Activity se asocia con exactamente un(a) Category.
- **Category** (0,*): cada Category puede asociarse con de cero a muchos(as) Activity.

4.3.3. Activity – ActivityType (Is of Type)

- **Activity** (1,1): cada Activity se asocia con exactamente un(a) ActivityType.
- **ActivityType** (0,*): cada ActivityType puede asociarse con de cero a muchos(as) Activity.

4.3.4. Category – Package (Coverage)

- **Category** (0,*): cada Category puede asociarse con de cero a muchos(as) Package.
- **Package** (1,*): cada Package se asocia con de uno a muchos(as) Category.

4.3.5. Activity – Guide (Generates)

- **Activity** (1,1): cada Activity se asocia con exactamente un(a) Guide.
- **Guide** (0,*): cada Guide puede asociarse con de cero a muchos(as) Activity.

4.3.6. Activity – Attraction (At)

- **Activity** (1,1): cada Activity se asocia con exactamente un(a) Attraction.
- **Attraction** (0,*): cada Attraction puede asociarse con de cero a muchos(as) Activity.

4.3.7. Guide – TravelGroup (Attends)

- **Guide** (0,*): cada Guide puede asociarse con de cero a muchos(as) TravelGroup.
- **TravelGroup** (1,1): cada TravelGroup se asocia con exactamente un(a) Guide.

4.3.8. Itinerary – Package (Belongs)

- **Itinerary** (1,1): cada Itinerary se asocia con exactamente un(a) Package.
- **Package** (0,*): cada Package puede asociarse con de cero a muchos(as) Itinerary.

4.3.9. Package – Admin (Generates)

- **Package** (1,1): cada Package se asocia con exactamente un(a) Admin.
- **Admin** (0,*): cada Admin puede asociarse con de cero a muchos(as) Package.

4.3.10. Itinerary – Guide (Generates)

- **Itinerary** (1,1): cada Itinerary se asocia con exactamente un(a) Guide.
- **Guide** (0,*): cada Guide puede asociarse con de cero a muchos(as) Itinerary.

4.3.11. Admin – Assignment (Assigns)

- **Admin** (0,*): cada Admin puede asociarse con de cero a muchos(as) Assignment.
- **Assignment** (1,1): cada Assignment se asocia con exactamente un(a) Admin.

4.3.12. Agregaciones

Las cardinalidades (min,max) de las doce agregaciones se recogen en la tabla de la sección 3.3. Destacan Proportion Type, Proportion Category e Itinerary Composition, donde el itinerario participa con cardinalidad mínima 1: todo itinerario debe tener al menos una proporción por tipo, una por categoría y al menos una actividad. En el resto, ambos extremos participan con (0,*), de modo que la existencia de la interrelación es opcional para ambas partes.

5. Valoración de la corrección del diseño de la base de datos

5.1. Demostración formal de normalización (3FN), PPDF, PLJ y discusión del diseño

Un diseño se considera teóricamente correcto si y solo si:

- Todos los esquemas relacionales de la descomposición están en Tercera Forma Normal (3FN) o superior.
- Se cumple la Propiedad de Preservación de Dependencias Funcionales (PPDF).
- Se cumple la Propiedad del Join sin Pérdida de Información (PLJ).

5.1.1. Extracción de dependencias funcionales

A partir de la especificación conceptual se aplica el algoritmo de extracción:

- **Entidades:** por cada conjunto de entidades con atributos $X \subseteq U$ y llave K se añade $K \to X$.
- **Interrelaciones:** por cada conjunto de entidades en un extremo de cardinalidad máxima 1 se añade $K - K_E \to K_E$, donde K_E es la llave de dicho conjunto. Así, de *Belongs* (Activity N:1 Category) se obtiene $ID_{Activity} \to ID_{Category}$, que se refleja en la llave foránea Activity.ID_Category.
- **Agregaciones y relaciones N:M:** por cada agregación con atributos propios X se añade $K \to X$, donde K es la concatenación de las llaves de los participantes (más el atributo discriminante de fecha cuando el histórico admite repeticiones).
- **Reglas de negocio:** se añaden las dependencias asociadas a restricciones explícitas del dominio.

5.1.2. Cubrimiento minimal

Un conjunto G de dependencias funcionales es un cubrimiento minimal de F si: (1) $G \equiv F$; (2) ningún determinante contiene atributos redundantes; y (3) G no contiene dependencias redundantes. Aplicando el algoritmo de minimización (descomposición del lado derecho y comprobación de redundancias vía clausura de atributos) se obtiene el conjunto irreducible que se muestra a continuación. En todos los casos el determinante es la llave completa de la relación: no existe ningún subconjunto propio que determine el resto de atributos (p. ej., en ItineraryValidation la fecha forma parte del determinante porque un mismo guía puede validar el mismo itinerario más de una vez).

Las dependencias triviales $K \to K$ que producen las relaciones cuyos atributos son todos llave (Admin, OperationsManager, Coverage) no forman parte del cubrimiento minimal; esos esquemas se conservan mediante el paso 3 del algoritmo de síntesis (atributos no contenidos en ninguna dependencia).

5.2. Descomposición formal en Tercera Forma Normal (3FN)

Para obtener la descomposición ρ en 3FN del esquema R(U, F), donde F es el cubrimiento minimal, se usa el algoritmo de síntesis:

- Reemplazar cada $X \to Y$ en F por las dependencias $X \to A_i$ para todo $A_i \in Y$.
- Por cada conjunto de dependencias $X \to A_1, \ldots, X \to A_k$ con el mismo determinante crear un esquema $R_j(U_j, F_j)$ con $U_j = X \cup \{A_1, \ldots, A_k\}$ y $F_j = \Pi_{Rj}(F)$.
- Si algún atributo de U no aparece en ninguna dependencia, formar con él un esquema propio (o integrarlo con su llave).
- Entonces $\rho = \{R_i\}$ está en 3FN respecto a $\Pi_{Ri}(F)$.

5.2.1. Aplicación del algoritmo

La aplicación del algoritmo agrupa las dependencias por determinante; como el cubrimiento tiene un único determinante por esquema, la descomposición coincide con los 25 esquemas del esquema lógico (sección 3.4). La siguiente tabla muestra, para cada uno, su llave candidata y la verificación de 3FN: todo atributo no primo depende directamente de la llave y no existe dependencia transitiva entre atributos no primos (las llaves foráneas son atributos no primos que no determinan a ningún otro atributo del esquema).

Esquema	**Llave**	**Atributos no primos**	**3FN**
R1 — User	ID	Name, Email	Sí (BCNF)
R2 — TravelGroup	ID	Name, ID_Guide	Sí (BCNF)
R3 — Package	ID	Name, Destination, Status, CreationDate, ID_Admin	Sí (BCNF)
R4 — Itinerary	ID	CreationDate, Status, ActivityCount, ID_Package, ID_Guide	Sí (BCNF)
R5 — ActivityType	Code	Name	Sí (BCNF)
R6 — Category	ID	Name	Sí (BCNF)
R7 — Attraction	ID	Name, Address, Latitude, Longitude	Sí (BCNF)
R8 — Activity	ID	Name, EffortLevel, Duration, Status, CreationDate, Code_Type, ID_Category, ID_Guide, ID_Attraction	Sí (BCNF)
R9 — Admin	ID	— (todos primos)	Sí (BCNF)
R10 — Guide	ID	Specialty	Sí (BCNF)
R11 — OperationsManager	ID	— (todos primos)	Sí (BCNF)
R12 — Tourist	ID	Age, ID_TravelGroup	Sí (BCNF)
R13 — Coverage	ID_Package, ID_Category	— (todos primos)	Sí (BCNF)
R14 — ProportionType	ID_Itinerary, Code_Type	Value	Sí (BCNF)
R15 — ProportionCategory	ID_Itinerary, ID_Category	Value	Sí (BCNF)
R16 — ItineraryValidation	ID_Itinerary, ID_Guide, ValidationDate	Result, Observations	Sí (BCNF)
R17 — ItineraryApproval	ID_Itinerary, ID_Manager, ApprovalDate	Result, Observations	Sí (BCNF)
R18 — PackageValidation	ID_Package, ID_Manager, ValidationDate	Result, Observations	Sí (BCNF)
R19 — ActivityValidation	ID_Activity, ID_Admin, ValidationDate	Result, Observations	Sí (BCNF)
R20 — Assignment	ID_Itinerary, ID_Tourist	AssignmentDate, Status, ID_Admin	Sí (BCNF)
R21 — ModificationRequest	ID_Itinerary, ID_Tourist, ID_Guide, RequestDate	Description, Status, Result	Sí (BCNF)
R22 — ItineraryComposition	ID_Itinerary, ID_Activity	Order, Day, StartTime	Sí (BCNF)
R23 — Visit	ID_Itinerary, ID_Tourist, ID_Activity	Comment, Rating, Status	Sí (BCNF)
R24 — Authorization	ID_Guide, ID_Package	CanGenerate, CanValidate	Sí (BCNF)
R25 — Booking	ID_Tourist, ID_Package	BookingDate	Sí (BCNF)

Todos los esquemas están además en la Forma Normal de Boyce-Codd (BCNF), puesto que el único determinante de cada uno es una llave candidata. Se asume que Email es único en User; de serlo, sería una segunda llave candidata y el esquema permanece en BCNF.

5.3. Garantía de cumplimiento de la PPDF y la PLJ

5.3.1. Preservación de dependencias funcionales (PPDF)

Por construcción del algoritmo de síntesis la PPDF se cumple siempre: cada dependencia X $\rightarrow$ Y $\in$ F dio origen a un esquema R_j con $U_j = X \cup Y$, de modo que $F \equiv \bigcup_{i=1..25} \Pi_{Ri}(F)$.

5.3.2. Join sin pérdida (PLJ)

Para evitar tuplas espurias al reconstruir la relación universal mediante reunión natural se aplica la condición de Ullman: la descomposición debe contener un esquema cuya llave sea llave candidata del esquema universal, o bien añadirse uno. Como alternativa constructiva, se verifica la propiedad por pares: la reunión de R_1 y R_2 es sin pérdida si $(U_1 \cap U_2) \rightarrow U_1$ o $(U_1 \cap U_2) \rightarrow U_2 \in F^+$. En este diseño toda reunión se realiza por una llave foránea contra la llave primaria del esquema referenciado (p. ej., Activity $\bowtie$ Guide por ID_Guide = Guide.ID, o Visit $\bowtie$ Assignment por (ID_Itinerary, ID_Tourist)), por lo que la intersección siempre determina al esquema referenciado. Aplicando esta condición de forma encadenada siguiendo el grafo acíclico de referencias (con User como raíz de la especialización y Package, Itinerary, Activity y Assignment como nodos intermedios) se obtiene una reunión sin pérdida de toda la descomposición. En caso de que se exija la formulación clásica, basta añadir el esquema $R_{26}(X)$ con X la llave candidata del universal, formada por la unión de las llaves de los esquemas anteriores.

5.4. Valoración final del diseño

5.4.1. Completitud respecto a los requerimientos

- **RF-01 a RF-07:** Activity, Attraction, ActivityType, Category y las relaciones Classifies, Is of Type y At; ActivityValidation soporta el control de duplicados.
- **RF-08 a RF-12:** Package, Coverage (categorías a cubrir), Authorization (generar/validar), Tourist, Guide, TravelGroup y Booking (inscripción a paquetes).
- **RF-13 a RF-18:** Itinerary, ProportionType, ProportionCategory (parametrización) y ItineraryComposition (orden, día y hora).
- **RF-19 a RF-21:** ItineraryValidation e ItineraryApproval con fecha, resultado y observaciones; el estado en Itinerary.
- **RF-22 a RF-28:** Assignment y Visit (comentario, valoración y estado de la visita).
- **RF-29 a RF-31:** ModificationRequest, con guía adicional y resultado.
- **RF-32 y RF-33:** fuera del esquema relacional, según RNF-02; el algoritmo de búsqueda de alternativas se describe en la sección 3.5.

De esta manera el modelo ofrece la base necesaria para implementar las consultas RC-1 a RC-6; en particular, la tasa de abandono de RC-6 se obtiene de Visit.Status, la traza hasta el guía creador de Activity.ID_Guide, y la comparación con el grupo de viaje de la cadena ModificationRequest $\rightarrow$ Assignment $\rightarrow$ Tourist $\rightarrow$ TravelGroup.

5.4.2. Observaciones y puntos abiertos detectados

Durante el análisis se detectaron los siguientes puntos que conviene resolver antes de escribir los modelos ORM:

- **PA-1. Administrador.** El esquema lo modela (Admin, Package.ID_Admin, Assignment.ID_Admin, ActivityValidation), pero el equipo acordó el 21 de septiembre no modelarlo como entidad. Queda pendiente decidir quién ejecuta RF-22 (asignación).

- **PA-2. User frente a Persona.** El profesor indicó que en el MERX debe existir Persona y no User, y que los permisos no se modelan; el revisor externo recomendó Usuario + Rol. Son criterios incompatibles y hay que elegir uno.
- **PA-3. Asignación doble.** El modelo solo impide asignar dos veces el mismo itinerario; impedir itinerarios solapados exige fechas por paquete o itinerario que el enunciado no pide.
- **PA-4 y PA-6. Grupo de viaje.** No está resuelto si el itinerario se asigna al turista o al grupo, ni si un turista pertenece a un único grupo (Tourist.ID_TravelGroup lo supone) o a uno por paquete.
- **PA-5.** Decisión interna no recogida en el enunciado: el turista toma el itinerario completo o lo cambia completo, sin modificación parcial.
- **PA-7. Autorización.** Authorization es un dato de negocio (qué paquetes atiende un guía) y debe distinguirse de los permisos de aplicación.
- **PA-8.** Los atributos de atracción, actividad, itinerario y grupo de viaje son decisiones del equipo no respaldadas por el enunciado.
- **PA-9.** ORM o SQL directo: se propone ORM para el CRUD y SQL escrito a mano para las seis consultas analíticas.

Además se señalan tres mejoras de diseño: (1) `Itinerary.ActivityCount` es un atributo derivado (cardinal de ItineraryComposition), redundancia que debe controlarse mediante restricción o vista; (2) `Order` es palabra reservada en SQL y conviene renombrarlo (p. ej., `Position`); (3) la llave (ID_Itinerary, ID_Activity) de ItineraryComposition impide repetir una actividad en un mismo itinerario, lo cual es coherente con la generación equilibrada pero debe confirmarse.

5.5. Flexibilidad y extensibilidad

El diseño presenta una estructura modular que facilita su adaptación ante la evolución del dominio sin afectar el esquema base. La independencia entre actividades, atracciones, categorías, tipos, paquetes e itinerarios permite incorporar nuevas combinaciones o paquetes de forma directa. La separación entre las agregaciones de validación (guía, jefe de operaciones, administrador) y las entidades permite incorporar nuevos revisores o nuevos roles sin alterar las relaciones existentes, y la gestión independiente de la red de atracciones permite evolucionarla (por ejemplo, hacia una base de datos orientada a grafos) sin modificar el modelo relacional.

6. Conclusiones

El desarrollo de este informe permitió consolidar una visión clara y teóricamente sólida del sistema de generación automática de itinerarios turísticos. La especificación de los requerimientos funcionales e informacionales garantiza que el sistema pueda responder a las seis consultas analíticas, al registro del orden de recorrido y a la trazabilidad entre actividades, guías, itinerarios, asignaciones y visitas.

La formulación del MERX, sumada a las restricciones de integridad para llaves, dominios y cardinalidades, refuerza la integridad operativa y evita estados inconsistentes. La aplicación de los algoritmos de síntesis sobre el cubrimiento minimal evidenció que el esquema lógico de 25 relaciones se encuentra en Tercera Forma Normal (incluso en BCNF) y cumple las propiedades PPDF y PLJ.

Queda pendiente cerrar los puntos abiertos señalados en la sección 5.4.2, en especial PA-1, PA-2 y PA-4, que condicionan la forma final del modelo. Una vez resueltos, el diseño estará preparado para escalar hacia su implementación física en PostgreSQL y su integración con el resto de la aplicación.